



UNIVERSITÀ DEGLI STUDI DI PADOVA

Corso di laurea in Informatica

Basi di Dati
Relazione di progetto

AppHub

Gestione dell'App Store

Candidato:

Gianmaria Grinovero

gianmaria.grinovero@studenti.unipd.it

Matricola: 2138007

Francesco Mazzurana

francesco.mazzurana@studenti.unipd.it

Matricola: 2149463

Anno Accademico 2025-2026

Indice:

1. Abstract:	1
2. Analisi dei requisiti:	1
3. Progettazione concettuale:	3
3.1 Schema E-R:	3
3.2 Dizionario:	4
4. Progettazione logica:	6
4.1 Analisi delle Ridondanze	6
4.2 Eliminazione gerarchie	8
4.4 Schema Relazionale	9
5. Implementazione in PostgreSQL e definizione delle Query	10
5.1 Definizione delle query	10
5.2 Definizione degli indici	13
6. Codice C	14

1. Abstract:

Il presente documento illustra la progettazione e l'implementazione di un sistema informativo dedicato all'amministrazione di uno store digitale per applicazioni mobili. Lo scopo del progetto è la creazione di un'infrastruttura dati solida, in grado di centralizzare l'intero catalogo dei software disponibili e di mappare tutte le interazioni operative e commerciali tra il pubblico e i creatori dei contenuti.

A livello funzionale, la piattaforma modella i prodotti informatici distinguendoli tra varianti gratuite, soggette a logiche di advertising, e versioni premium, regolamentate da periodi di prova e acquisti in-app. Vengono inoltre schematizzati i vari attori dell'ecosistema: da una parte i programmatori che pubblicano i titoli, comprensivi delle relative informazioni fiscali e di fatturazione, e dall'altra i clienti finali che navigano la vetrina tramite i propri dispositivi fisici. Il database storicizza i download eseguiti, le recensioni testuali rilasciate dai consumatori e tiene conto dei vincoli operativi, come le versioni minime dei sistemi operativi necessarie per garantire la corretta installazione.

L'architettura dei dati qui proposta mira a massimizzare l'affidabilità nella memorizzazione dei log e delle transazioni, consentendo al contempo interrogazioni rapide per l'estrazione di metriche di business (es. numero di download o ricavi). In questo modo, l'impianto relazionale fornisce una base concreta per gestire il ciclo di vita delle app e supportare le esigenze direzionali del marketplace

2. Analisi dei requisiti:

Il sistema gestisce un marketplace digitale per applicazioni mobili. Le specifiche dei dati da memorizzare sono strutturate come segue:

Sviluppatori: ogni sviluppatore che pubblica applicazioni sulla piattaforma è identificato da un codice univoco e contiene le seguenti informazioni:

- **ID Sviluppatore** che distingue in modo univoco il programmatore o l'azienda.
- Il **Nome** dello sviluppatore.
- L'indirizzo **Email**
- Il **Sito Web** ufficiale.

Per poter ricevere i pagamenti, ogni sviluppatore deve obbligatoriamente registrare un unico profilo fiscale associato.

Profilo Fiscale: ogni profilo fiscale contiene le seguenti informazioni:

- La **Partita IVA**, che identifica univocamente il profilo.
- Il codice **IBAN** per i versamenti.
- La **Nazione** di residenza fiscale.

Applicazioni: le applicazioni costituiscono il catalogo dello store. Ogni applicazione è identificata da un codice univoco e contiene le seguenti informazioni:

- L'**ID App** che distingue in modo univoco l'applicazione.
- Il **Nome** commerciale dell'applicazione.
- La **Versione** attuale del software.
- La **Dimensione** occupata in megabyte.
- La **Data** del primo **rilascio**.
- Il **Numero di download** totali effettuati.

Ogni applicazione è pubblicata da un solo sviluppatore e appartiene ad almeno una categoria tematica. Le applicazioni si suddividono in: gratuite e premium.

App Gratuite: oltre alle informazioni generali, per le applicazioni gratuite vengono registrati ulteriori dati:

- La presenza o meno di **Banner** pubblicitari (valore booleano).
- La **Frequenza** degli annunci pubblicitari.

App Premium: per le applicazioni a pagamento, oltre alle informazioni di base, si registra:

- Il **Prezzo** di acquisto.
- I **Giorni di prova** gratuita concessi all'utente.

Categorie: le categorie permettono di classificare le applicazioni e contengono le seguenti informazioni:

- Il **Nome** della categoria, che la identifica univocamente all'interno del sistema.

Utenti: I clienti dello store sono gli utenti, identificati da un nickname univoco, per i quali si registra:

- Il **Nickname** che distingue in modo univoco ogni utente.
- L'indirizzo **Email**.
- La **Data di Iscrizione** alla piattaforma.

Un dispositivo appartiene a un solo utente, ma un utente può possedere più dispositivi.

Dispositivi: i terminali fisici utilizzati per accedere allo store sono identificati in modo univoco e contengono le seguenti informazioni:

- Il codice **IMEI** che distingue univocamente il dispositivo hardware.
- La **Marca** produttrice.
- Il **Modello** del dispositivo.

Inoltre, ogni dispositivo opera su una specifica piattaforma.

Piattaforme: le piattaforme rappresentano i sistemi operativi supportati. Esse contengono le seguenti informazioni:

- Il **Nome** del sistema operativo, che lo identifica univocamente.
- Il nome del **Produttore**.

Un'applicazione può essere compatibile con più piattaforme, e ciascuna piattaforma ospita molteplici applicazioni. Per ogni associazione di compatibilità si registra la versione minima del sistema operativo richiesta per il corretto funzionamento.

Download: i download rappresentano l'evento di scaricamento di un'applicazione. È identificato esternamente dall'**utente** che lo effettua, dall'**applicazione** scaricata e dalla **data e ora** in cui avviene. Lo stesso utente può scaricare la stessa app più volte in date diverse e su dispositivi diversi. Tuttavia, un download può avvenire solo se la piattaforma del dispositivo è compatibile con l'applicazione e la versione del sistema operativo installata soddisfa la versione minima richiesta.

Transazioni: gli acquisti effettuati dagli utenti sono registrati come transazioni, le quali sono identificate da un codice univoco e contengono le seguenti informazioni:

- Il **Codice** alfanumerico univoco della transazione.
- L'**Importo** effettivamente pagato.
- La **data di pagamento**.
- Il **metodo di pagamento** utilizzato.

Le transazioni sono associate esclusivamente all'acquisto di un'App Premium da parte di un Utente.

Recensioni: le recensioni lasciate dagli utenti sono identificate dalla combinazione dell'**utente** autore e dell'**applicazione** valutata. Esse contengono le seguenti informazioni:

- Il **Voto** numerico assegnato.
- La **Data di pubblicazione** della recensione.
- Il **Testo** descrittivo.

3. Progettazione concettuale

3.1 Schema E-R:

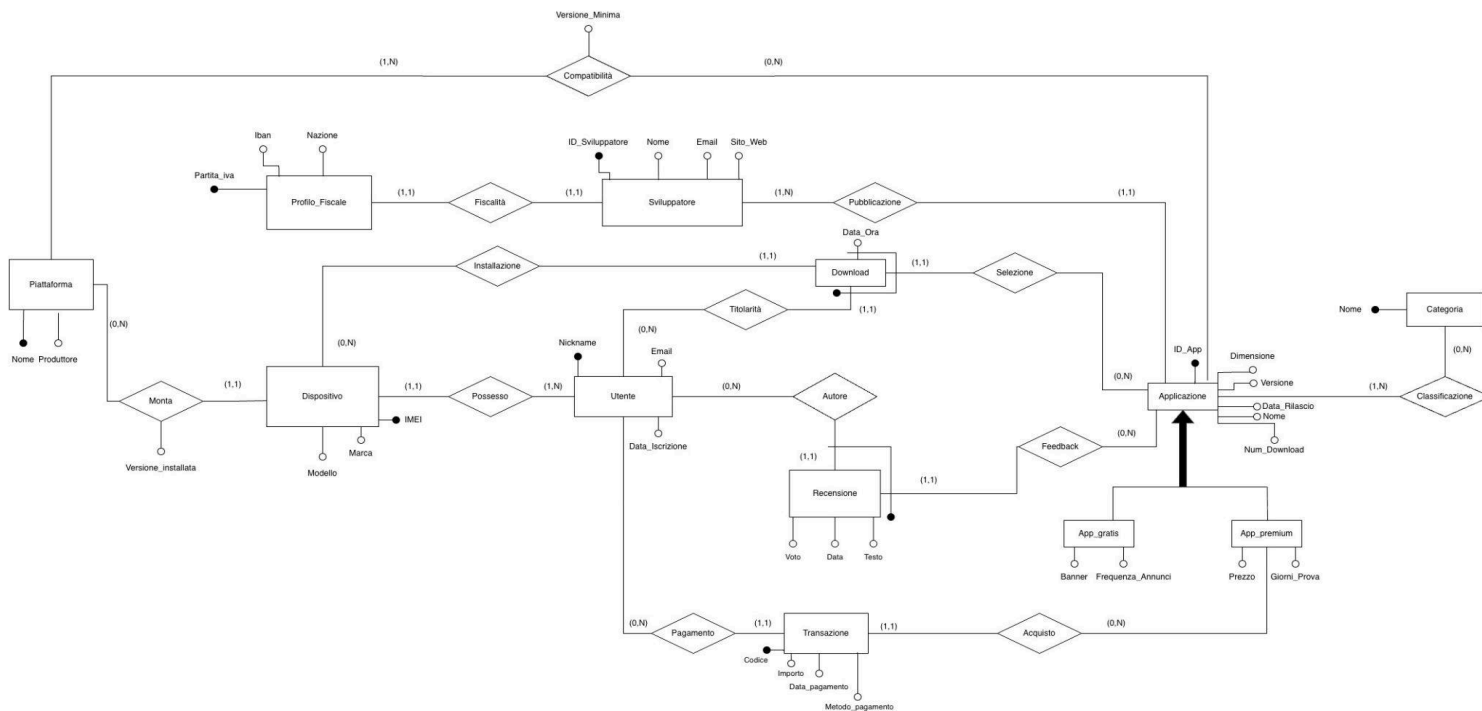


Figura 1: Schema E-R

La Figura 1 mostra il diagramma Entità-Relazione che riassume i requisiti dell'app store. Il catalogo prevede esclusivamente due tipologie di applicazioni: gratuite e premium. A livello concettuale, questa distinzione è modellata tramite una gerarchia totale ed esclusiva avente *Applicazione* come entità padre. Poiché l'analisi dei requisiti richiede il tracciamento delle vendite per le app a pagamento, la relazione *Acquisito* è stata collegata in modo diretto e nativo all'entità figlia *App_Premium*. Per quanto riguarda le interazioni degli utenti con lo store, ovvero le azioni di *Download* e *Recensione*, sono state modellate come entità deboli. Essendo prive di un proprio identificatore autonomo, la loro univocità dipende logicamente dalle entità forti a cui si appoggiano. In particolare ogni istanza viene infatti identificata dalla combinazione dell'*Utente* che esegue l'azione e dell'*Applicazione* coinvolta, con l'integrazione del *timestamp* per distinguere i download. Per rappresentare i sistemi operativi è stata introdotta l'entità indipendente *Piattaforma*. Questa soluzione consente di strutturare in modo pulito due associazioni multi-a-molti (N:N) fondamentali: da un lato la compatibilità con le applicazioni, definita dall'attributo *Versione_Minima*, dall'altro l'installazione sui dispositivi caratterizzata dall'attributo *Versione_installata*. La presenza di questi attributi sulle relazioni fornisce i vincoli necessari per poter validare correttamente l'esecuzione dei download.

Vincolo esterno (non esprimibile nello schema E-R):

Un'applicazione può essere scaricata su un dispositivo solo se il dispositivo monta una piattaforma compatibile con l'app, e se la versione del sistema operativo installata è maggiore o uguale alla versione minima richiesta.

In notazione formale rigorosa:

$\forall d \in \text{Download}, \forall a \in \text{Applicazione}, \forall disp \in \text{Dispositivo}, \forall p \in \text{Piattaforma}:$
 $(d, a) \in \text{Selezione} \wedge (d, disp) \in \text{Installazione} \wedge (disp, p) \in \text{Monta}$

$\Rightarrow (a, p) \in \text{Compatibilità} \wedge \text{Monta}(\text{disp}, p). \text{Versione_installata} \geq \text{Compatibilità}(a, p). \text{Versione_Minima}$

Individuazione ridondanza:

L'attributo *Num_Download* in *Applicazione* rappresenta una ridondanza.

Questo valore può essere ottenuto contando il numero di occorrenze della relazione *Selezione* associate a quella specifica applicazione.

L'analisi dei costi verrà affrontata nella successiva fase di progettazione logica

3.2 Dizionario:

Tabella entità:

Entità	Descrizione	Attributi	Identificatore
Sviluppatore	Azienda o programmatore che pubblica App	ID_Sviluppatore, Nome, Email, Sito_Web	ID_Sviluppatore
Profilo_Fiscale	Dato per il pagamento	Partita_Iva, Iban, Nazione	Partita_Iva
Applicazione	Applicazione presente nello store	ID_App, Dimensione, Versione, Data_Rilascio, Nome, Num_Download	ID_App
App_gratis	App gratuite	Banner, Frequenza_Annunci	
App_premium	App a pagamento	Prezzo, Giorni_Prova	
Categoria	Tipi di applicazione	Nome	Nome
Utente	Persona iscritta all'AppStore	Nickname, Email, Data_Iscrizione	Nickname
Dispositivo	dispositivo fisico	IMEI, Marca, Modello,	IMEI
Transazione	evento di pagamento	Codice, Importo, Data_Pagamento, Metodo_Pagamento	Codice
Recensione	evento di scrittura di una recensione	Voto, Data, Testo	Relazione "Autore", Relazione "Feedback"
Download	evento di scaricamento di un'app	Data_Ora,	Relazione Selezione Relazione Titolarità Data_ora
Piattaforma	sistema operativo	Nome, produttore	Nome

Tabella relazioni:

Relazioni	Descrizione	Componente	Attributi
Fiscalità	Associazione tra uno sviluppatore e il suo profilo fiscale per la gestione dei pagamenti	Profilo_Fiscale (1,1), Sviluppatore (1,1)	
Pubblicazione	Pubblicazione di un'app sullo store da parte di uno sviluppatore	Sviluppatore (1,N), Applicazione (1,1)	
Classificazione	Assegnazione di un'app a una o più categorie	Applicazione (1,N), Categoria (0,N)	
Possesso	Associazione di un dispositivo fisico ad un utente	Dispositivo (1,1), Utente (1,N)	
Titolarità	Associazione che indica quale utente ha effettuato il download	Utente (0,N) Download (1,1)	
Selezione	Collegamento tra un evento di download e l'applicazione specifica scaricata	Download (1,1), Applicazione (0,N)	
Installazione	Associazione che indica su quale dispositivo è avvenuto determinato download	Dispositivo (0,N), Download (1,1)	
Acquisto	Associazione tra una transazione economica e l'app premium acquistata	Transazione (1,1), App_Premium (0,N)	
Autore	Collegamento tra una recensione e l'utente che l'ha scritta	Utente (0,N), Recensione (1,1)	
Pagamento	Collegamento tra una transazione economica e l'utente che ha effettuato il pagamento	Utente (0,N) Transazione(1,1)	
Feedback	Associazione tra una recensione e l'applicazione	Recensione (1,1), Applicazione (0,N)	

Monta	Associa il dispositivo alla versione di sistema operativo installato	Dispositivo (1,1), Piattaforma (0,N)	Versione_Installata
Compatibilità	Indica le piattaforme su cui una determinata applicazione può essere eseguita	Piattaforma ((1,N), Applicazione (0,N)	Versione_Minima

4. Progettazione logica:

4.1 Analisi delle Ridondanze

L'attributo '*Num_Download*' nell'entità *Applicazione* rappresenta una ridondanza, poiché tale valore può essere calcolato derivando il numero di occorrenze della relazione *Selezione* (o dall'entità *Download*) associate a una specifica applicazione.

Per decidere se mantenere o eliminare l'attributo, effettuiamo un'analisi del costo delle operazioni coinvolte, ipotizzando i seguenti volumi attesi per la base di dati:

Concetto	Costrutto	Volume
Applicazione	Entità (E)	10.000
Utente	Entità (E)	50.000
Download	Entità (E)	1.000.000
Selezione	Relazione (R)	1.000.000
Piattaforma	Entità (E)	5
Compatibilità	Relazione(R)	25.000

Le operazioni critiche individuate nell'analisi dei requisiti sono:

- **Operazione 1 (500 al giorno):** Registra un nuovo download di un'applicazione da parte di un utente
- **Operazione 2 (100 al giorno):** Visualizza il numero totale di download di una specifica applicazione per mostrare le classifiche

Nota: Assumiamo che il costo di un accesso in scrittura (S) sia doppio rispetto a un accesso in Lettura (L);

Costo Senza Ridondanza:

Analizziamo il costo totale rimuovendo l'attributo '*Num_Download*', che rende necessario il calcolo dinamico del valore. In questo scenario, per ottenere il totale dei download, serve scorrere tutte le istanze della relazione *Selezione* ad essa ad essa associate. Il numero medio di download per app è pari al rapporto tra il volume dei download e il volume delle app $1.000.000/10.000 = 100$.

-Operazione 1: *Nota:* prima di registrare il download, si verifica il vincolo esterno di compatibilità leggendo il Dispositivo e la relazione di Compatibilità;

Concetto	Costrutto	Accessi	Tipo	Frequenza
<i>Dispositivo</i>	E	1	Lettura	500/g
<i>Compatibilità</i>	R	1	Lettura	500/g
<i>Download</i>	E	1	Scrittura	500/g
<i>Titolarità (Utente)</i>	R	1	Scrittura	500/g
<i>Selezione (Applicazione)</i>	R	1	Scrittura	500/g
<i>Installazione (Dispositivo)</i>	R	1	Scrittura	500/g

-Operazione 2:

Concetto	Costrutto	Accessi	Tipo	Frequenza
<i>Applicazione</i>	E	1	Lettura	100/g
<i>Selezione</i>	R	100	Lettura	100/g

Costo Totale Senza Ridondanza:

- Costo Operazione 1: $[(2 * 1) + (4 * 2)] * 500 = 5.000$
- Costo Operazione 2: $(101 * 1) * 100 = 10.100$

Totale: 15.100

Costo con Ridondanza:

Analizziamo il costo mantenendo l'attributo 'Num_Download' nell'entità *Applicazione*. In questo caso, ogni nuovo download richiede l'aggiornamento (in scrittura) del contatore dell'app, ma la lettura del totale è immediata.

-Operazione 1: Nota: verifica vincolo compatibilità inclusa;

Concetto	Costrutto	Accessi	Tipo	Frequenza
<i>Dispositivo</i>	E	1	Lettura	500/g
<i>Compatibilità</i>	R	1	Lettura	500/g
<i>Download</i>	E	1	Scrittura	500/g
<i>Titolarità (Utente)</i>	R	1	Scrittura	500/g
<i>Selezione (Applicazione)</i>	R	1	Scrittura	500/g
<i>Installazione (Dispositivo)</i>	R	1	Scrittura	500/g
<i>Applicazione</i>	E	1	Lettura	500/g
<i>Applicazione</i>	E	1	Scrittura	500/g

-Operazione 2:

Concetto	Costrutto	Accessi	Tipo	Frequenza
Applicazione	E	1	Lettura	100/g

Costo Totale con Ridondanza:

- Costo Operazione 1: $[(3 * 1) + (5 * 2)] * 500 = 13 * 500 = 6.500$
 - Costo Operazione 2: $(1 * 1) * 100 = 100$
- Totale: 6.600**

Conclusione:

Come si evince dall'analisi, il costo giornaliero delle operazioni in presenza di ridondanza è nettamente inferiore al costo calcolato in assenza di essa.

Pertanto, si sceglie di mantenere l'attributo ridondante '*Num_Download*' nell'entità *Applicazione*.

Nonostante il vincolo esterno di compatibilità, pur aumentando leggermente i costi fissi di scrittura a causa delle letture preventive, non altera la convenienza della ridondanza.

4.2 Eliminazione gerarchie

Nello schema concettuale originario è presente una generalizzazione di tipo totale ed esclusiva per l'entità *Applicazione*, la quale si specializza nelle due entità figlie *App_Gratuita* e *App_Premium*. Per ottimizzare il modello logico-relazionale si procede all'eliminazione di questa gerarchia attraverso l'accorpamento delle entità figlie nell'entità padre *Applicazione*.

Questa specifica scelta implementativa è stata adottata per i seguenti motivi:

- Riduzione della complessità dello schema:** Le entità figlie possiedono pochi attributi specifici ('*Ha_Banner*' e '*Frequenza_Annunci*' per le app gratuite mentre '*Prezzo*' e '*Giorni_Prova*' per le app premium). Mantenere tabelle separate per due soli campi creerebbe frammentazione inutile. L'accorpamento riduce il numero di entità nel database e semplifica la manutenzione.
- Ottimizzazione delle query:** L'operazione più frequente all'interno di un app store è la ricerca e la visualizzazione del catalogo completo delle applicazioni, sia gratuite che a pagamento. Mantenendo un'unica tabella *Applicazione* non sarà necessario effettuare costose operazioni di unione tra tabelle separate per mostrare la lista globale delle app.
- Uso di attributo discriminante:** Per preservare la distinzione logica tra le due categorie, viene introdotto un attributo enumerativo discriminante '*Tipo*', il quale assumerà i valori testuali '*Gratuita*' o '*Premium*'. Gli attributi non pertinenti alla tipologia specifica dell'istanza assumeranno valore *NULL*.
- Prevenzione dello sdoppiamento delle relazioni:** Si è scartata l'ipotesi opposta, ovvero il collassamento dell'entità padre verso le figlie, poiché *Applicazione* rappresenta il nucleo centrale del dominio. L'eliminazione del padre avrebbe costretto a sdoppiare in modo ridondante e inefficiente tutte le relazioni ad essa collegate, generando inoltre una grave duplicazione degli attributi comuni a tutte le app.
- Gestione della relazione Acquisto:** Nello schema concettuale, la relazione Acquisto coinvolge nativamente solo l'entità figlia *App_Premium*. Avendo collassato la gerarchia verso l'alto, la chiave esterna di *Transazione* punterà alla tabella padre *Applicazione*. La garanzia che le transazioni siano collegate esclusivamente ad App Premium viene demandata alla logica applicativa esterna (oppure verrebbe gestita tramite un Trigger dedicato, non visto a lezione e dunque non implementato).

Questo approccio garantisce la massima efficienza in lettura senza sacrificare l'integrità concettuale dei dati originari a discapito della presenza di alcuni campi *NULL*.

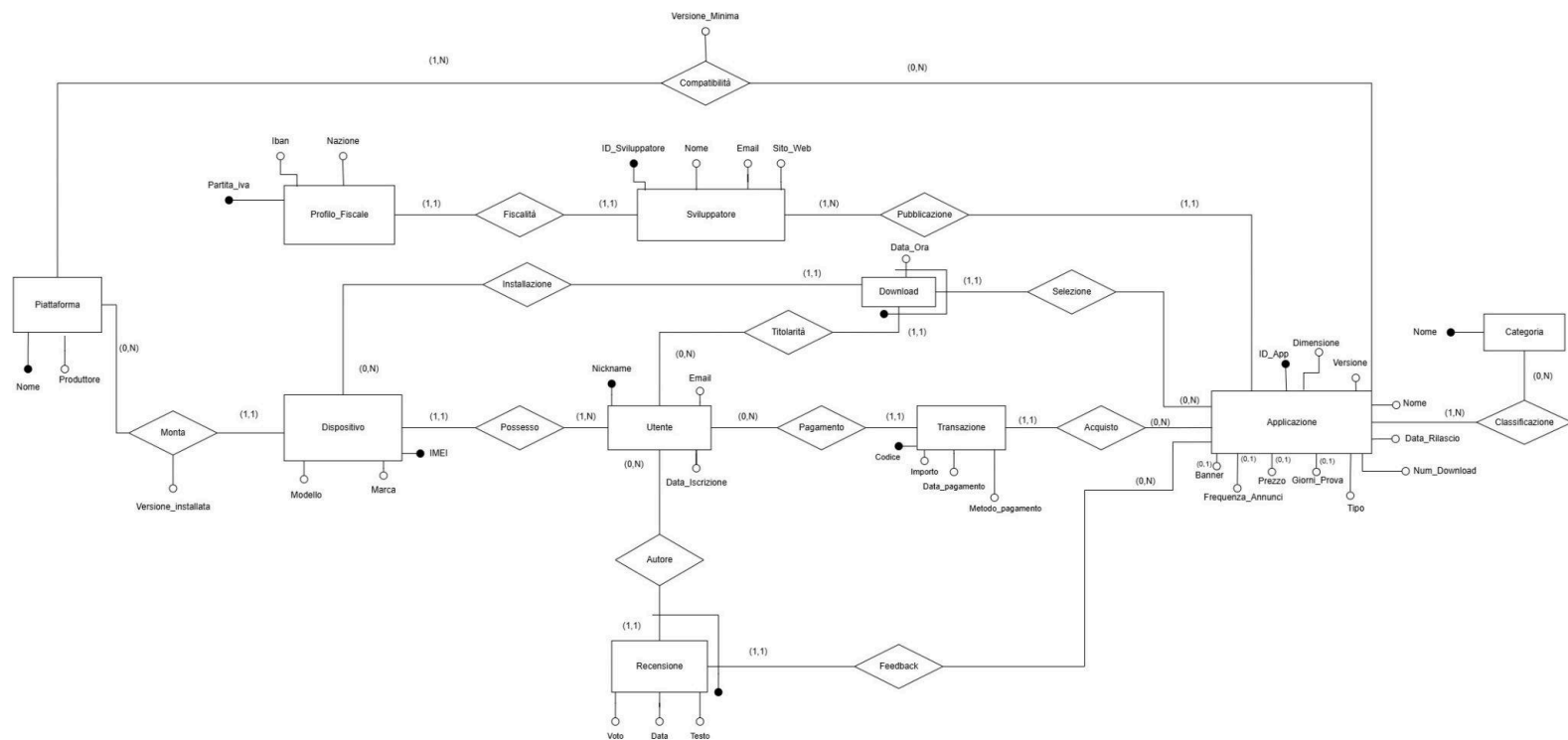


Figura 2: Schema E-R ristrutturato.

4.4 Schema Relazionale

Nota: L'asterisco * dopo il nome degli attributi indica l'ammissibilità di valori NULL;

Profilo_Fiscale(Partita_iva, Iban, Nazione)

Sviluppatore(ID_Sviluppatore, Nome, Email, Sito_web, Fiscalità)

→ Sviluppo.Fiscalità → Profilo_Fiscale.Partita_iva

Applicazione(ID_App, Sviluppatore, Dimensione, Versione, Nome, Data_Rilascio, Num_Download, Tipo, Giorni_Prova*, Prezzo*, Frequenza_Annunci*, Banner*)

→ Applicazione.Sviluppatore → Sviluppatore.ID_Sviluppatore

Categoria(Nome)

Classificazione(Applicazione, Categoria)

Classificazione.Applicazione → Applicazione.ID_App

Classificazione.Categoria → Categoria.Nome

Utente(Nickname, Email, Data_Iscrizione)

Recensione(Utente, Applicazione, Voto, Data, Testo)

→ Recensione.Utente → Utente.Nickname

→ Recensione.Applicazione → Applicazione.ID_App

Transazione(Codice, Utente, Applicazione, Importo, Data_Pagamento, Metodo_Pagamento)

→ Transazione.Utente → Utente.Nickname

→ Transazione.Applicazione → Applicazione.ID_App

Piattaforma(Nome, Produttore)

Dispositivo(IMEI, Utente, Marca, Modello, Piattaforma, Versione_installata)

- Dispositivo.Utente→Utente.Nickname
- Dispositivo.Piattaforma→Piattaforma.Nome

Download(Utente, Applicazione, Dispositivo, Data_Ora)

- Download.Utente→Utente.Nickname
- Download.Applicazione→Applicazione.ID_App
- Download.Dispositivo→Dispositivo.IMEI

Compatibilità(Piattaforma, Applicazione, Versione_Minima)

- Compatibilità.Piattaforma→Piattaforma.Nome
- Compatibilità.Applicazione→Applicazione.ID_App

5. Implementazione in PostgreSQL e definizione delle Query

Il file appHub.sql contiene il codice SQL necessario per la creazione e il popolamento delle tabelle del database. Il file in questione contiene una serie di query per l'estrazione dei dati e degli indici creati per migliorare le prestazioni di alcune interrogazioni.

5.1 Definizione delle query

Query 1: Visionare lo storico completo dei download ordinati per data effettuati da uno specifico utente in input (ad es. 'marco_92') per ogni suo dispositivo;

```
1 SELECT A.Nome as nomeApplicazione, Di.marca, di.modello as Dispositivo, A.dimensione AS Dimensione_MB,
2      d.data_ora as Orario_Download
3 FROM Applicazione A JOIN Download D ON A.id_app = D.applicazione
4 JOIN Dispositivo Di ON Di.imei= D.dispositivo
5 WHERE D.utente='marco_92'
6 ORDER BY D.data_ora DESC;
```

	nomeapplicazione character varying (100) 🔒	marca character varying (50) 🔒	dispositivo character varying (100) 🔒	dimensione_mb numeric (8,2)	orario_download timestamp without time zone 🔒
1	GiocoCarte	Microsoft	Surface Pro 9	65.00	2025-04-01 12:00:00
2	StudioVideo	Samsung	Galaxy S24	180.00	2025-03-15 17:00:00
3	CloudDrive	Apple	iPhone 15 Pro	28.30	2025-02-20 09:30:00
4	SocialVIP	Microsoft	Surface Pro 9	38.90	2025-01-10 13:00:00
5	TaskMaster	Samsung	Galaxy S24	32.50	2024-11-15 10:00:00
6	ProEditor	Apple	iPhone 15 Pro	120.00	2024-10-01 08:45:00
7	SocialPost	Samsung	Galaxy S24	28.40	2024-09-05 16:30:00
8	MappaViaggi	Microsoft	Surface Pro 9	42.70	2024-08-10 11:00:00
9	GiocoQuiz	Apple	iPhone 15 Pro	78.00	2024-07-20 14:00:00
10	FotoSnap	Samsung	Galaxy S24	45.50	2024-06-15 09:30:00

Query 2: Elencare gli sviluppatori specificando anche la loro nazione di residenza fiscale e i loro guadagni medi da ogni app pubblicata premium;

```

4 CREATE VIEW Guadagni_APP(id_sv, id_app, totale) as
5 SELECT sviluppatore, id_app, SUM(T.importo)
6 FROM Applicazione A JOIN Transazione T ON A.id_app=T.applicazione
7 GROUP BY A.sviluppatore, A.id_app;
8
9 CREATE VIEW Media_sviluppatore(id_sviluppatore, media) AS
10 SELECT id_sv, ROUND(AVG(totale),2)
11 FROM Guadagni_APP
12 GROUP BY id_sv;
13
14 SELECT DISTINCT S.Nome, PF.nazione, M.media AS Guadagno_Medio
15 FROM Profilo_fiscale PF JOIN Sviluppatore S ON PF.partita_iva=S.fiscalita
16 JOIN Media_sviluppatore M ON S.id_sviluppatore=M.id_sviluppatore
17 ORDER BY M.media desc;

```

	nome character varying (100)	nazione character (2)	guadagno_medio numeric
1	AppVision Srl	IT	44.97
2	CloudNet GmbH	DE	39.95
3	InnovaCode SpA	IT	35.97
4	EuroTech Solutions	IT	35.94
5	AlphaCode Srl	IT	23.22
6	CodeMasters Italia	IT	20.97
7	ByteForge Srl	IT	19.98
8	ProDev Solutions	IT	13.48
9	TechnoApp Srl	IT	10.47
10	Digital Dreams SpA	IT	9.96
11	SmartSoft Italia	IT	8.98
12	MobileFirst SpA	IT	7.98
13	NexGen Apps	IT	5.98
14	DataFlow SAS	FR	3.96
15	CreativeApps Srl	IT	2.98

Query 3: Per ogni Categoria elencare il numero di applicazioni, il numero totale di download delle applicazioni appartenenti e il ricavo medio dalle Premium;

```

1 CREATE VIEW Ricavi_Per_App(applicazione, ricavo) AS
2 SELECT Applicazione, SUM(Importo)
3 FROM Transazione
4 GROUP BY Applicazione;
5
6 SELECT cl.categoria AS Categoria, COUNT(a.ID_App) AS Numero_App,
7 SUM(a.Num_Download) AS Download_Totali,
8 ROUND(AVG(r.Ricavo), 2) AS Ricavo_Medio_Per_App
9 FROM Classificazione cl JOIN Applicazione a ON cl.Applicazione = a.ID_App
10 LEFT JOIN Ricavi_Per_App r ON a.ID_App = r.Applicazione
11 GROUP BY cl.categoria
12 ORDER BY Download_Totali DESC;

```

	categoria character varying (50) 🔒	numero_app bigint 🔒	download_totali bigint 🔒	ricavo_medio_per_app numeric 🔒
1	Intrattenimento	7	32	20.64
2	Produttività	6	26	21.72
3	Fotografia	6	25	24.64
4	Giochi	6	22	13.47
5	Educazione	5	19	10.30
6	Fitness	5	18	16.48
7	Stile di vita	5	18	2.98
8	Social	3	17	35.94
9	Finanza	4	17	18.63
10	Musica	4	16	8.48
11	Notizie	3	14	[null]
12	Viaggi	2	8	20.97

Nota: Se una categoria non ha app Premium, o se le sue app Premium non hanno generato transazioni, comparirà *NULL*;

Query 4: Trovare la "Top X"(nello specifico esempio 5) delle applicazioni più "apprezate" e recensite nello store. Più nello specifico: le applicazioni che hanno ricevuto almeno N (nello esempio specifico N=1) recensioni, ordinate per voto medio decrescente e, a parità di voto, per numero di recensioni;

```

1  SELECT Nome AS Nome_app, COUNT(*) AS num_recensioni, ROUND(AVG(voto),2) AS voto_medio
2  FROM Applicazione A JOIN Recensione R ON A.id_app=R.applicazione
3  GROUP BY A.id_app
4  HAVING COUNT(*)>=1
5  ORDER BY voto_medio DESC, COUNT(*) DESC
6  Limit 5

```

	nome_app character varying (100) 🔒	num_recensioni bigint 🔒	voto_medio numeric 🔒
1	ViaggioPlus	2	5.00
2	ChatAmici	3	4.67
3	TaskMaster	3	4.67
4	ScuolaPlus	2	4.50
5	FitnessPro	2	4.50

Query 5: Per ciascuna Piattaforma calcola il numero di applicazioni compatibili, i dispositivi registrati e il volume totale di download;

```

1 CREATE VIEW App_Per_Piattaforma(piattaforma, num_app) AS
2 SELECT Piattaforma, COUNT(*)
3 FROM Compatibilita
4 GROUP BY Piattaforma;
5
6 CREATE VIEW Dispositivi_Download(piattaforma, num_dispositivi, num_download) AS
7 SELECT d.Piattaforma, COUNT(DISTINCT d.IMEI), COUNT(dl.Data_Ora)
8 FROM Dispositivo d
9 LEFT JOIN Download dl ON d.IMEI = dl.Dispositivo
10 GROUP BY d.Piattaforma;
11
12 SELECT p.Nome AS Piattaforma,
13        p.Produttore,
14        ap.Num_App AS App_Compatibili,
15        dd.Num_Dispositivi AS Dispositivi_Registrati,
16        dd.Num_Download AS Download_Totali
17 FROM Piattaforma p
18 JOIN App_Per_Piattaforma ap ON p.Nome = ap.Piattaforma
19 JOIN Dispositivi_Download dd ON p.Nome = dd.Piattaforma
20 ORDER BY Download_Totali DESC;

```

	piattaforma character varying (50) 🔒	produttore character varying (100) 🔒	app_compatibili bigint 🔒	dispositivi_registrati bigint 🔒	download_totali bigint 🔒
1	Android	Google	33	22	62
2	iOS	Apple	34	19	49
3	Windows	Microsoft	16	10	27
4	HarmonyOS	Huawei	25	10	26

5.2 Definizione degli indici

Si è deciso di migliorare le performance delle query del sistema attraverso la creazione di indici mirati. Come regola generale, non è stato necessario creare indici sulle chiavi primarie delle tabelle poiché PostgreSQL crea automaticamente un indice B+Tree per ogni primary key, mantenendone un ordinamento che rende già ottimali le ricerche dirette su tali campi.

-ottimizzazione query 4:

Per questa query occorre considerare:

- Condizione di Join tra Applicazione e Recensione: `A.id\_app = R.applicazione`;
- Group by sulla colonna `A.id\_app` e aggregazione dei voti.

Il Join richiede l'accesso alle tuple di Recensione basate sull'app. Poiché la chiave primaria di Recensione è `(Utente, Applicazione)`, la ricerca per sola applicazione risulta inefficiente. Essendo una ricerca per uguaglianza esatta, è opportuno creare un indice Hash:

```
CREATE INDEX idx_recensione ON Recensione USING HASH (Applicazione);
```

Questo permette di fare il join usando i codici hash, che è più efficiente. L'operazione di *Group by* su 'A.id_app' invece, sfrutta l'indice B+Tree primario già esistente in PostgreSQL.

-ottimizzazione query 2 e 3:

In entrambe le query occorre considerare:

- Condizione di Join tra Applicazione e Transazione: 'A.id_app = T.applicazione';
- Group By sulla colonna 'T.applicazione';

L'operazione richiede di raggruppare tutte le transazioni appartenenti alla stessa applicazione. Per ottimizzare sia il Join che l'ordinamento necessario al *Group by*, viene creato il seguente indice B+Tree:

```
CREATE INDEX idx_transazione_app ON Transazione(Applicazione);
```

-ottimizzazione query 5:

Per calcolare correttamente i volumi per piattaforma, la query effettua un Left-Join:

- Condizione di Join tra Dispositivo e Download: 'dl.Dispositivo = d.IMEI'.

Il sistema deve accedere alla tabella 'Download' partendo dall'identificativo di *Dispositivo*. Poiché la chiave primaria di *Download* è composta da '(Utente, Applicazione, Data_Ora)', una ricerca basata unicamente sul dispositivo scatenerrebbe una scansione sequenziale dell'intera tabella. Per abbattere il costo di accesso, si implementa un indice B+Tree secondario:

```
CREATE INDEX idx_download_dispositivo ON Download(Dispositivo);
```

-ottimizzazione query 1:

Per questa query occorre considerare:

- Condizione di selezione sull'utente: es. 'D.utente='marco_92' '.
- Ordinamento sulla colonna 'D.data_ora'.

La ricerca richiede di isolare i download di uno specifico utente per poi disporli in sequenza temporale. Nonostante la chiave primaria '(Utente, Applicazione, Data_Ora)' garantisca già un accesso rapido tramite il primo campo, la presenza di 'Applicazione' nel mezzo della chiave spezza l'ordinamento richiesto. Per eliminare queste operazioni extra e accelerare il recupero dei dati, viene definito il seguente indice B+Tree composito:

```
CREATE INDEX idx_download_utente_data ON Download(Utente, Data_Ora);
```

6. Codice C

Il file AppHub.c contiene il codice necessario per accedere ed interfacciarsi in modo sicuro e interattivo alla base di dati dell'AppHub. Il codice può essere compilato ed eseguito tramite i seguenti comandi (assumendo la presenza delle librerie necessarie nella cartella dependencies)

```
gcc AppHub.c -L dependencies/lib -lpq -o apphub
```

```
./apphub
```

All'avvio, il programma presenta una "dashboard" interattiva che permette di configurare agilmente i parametri di connessione al database in Postgres (User, Password, Nome DB, IP Host, Porta), offrendo un'interfaccia a riga di comando curata graficamente.

Una volta stabilita la connessione, viene mostrato all'utente un menù principale con le seguenti opzioni:

- **[0]** Stampa lista query
- **[1-5]** Esecuzione delle rispettive query
- **[6]** Esci dal programma

Tramite questo prompt (stilizzato come [AppHub] >) l'utente può scegliere di interrogare il database, stampare nuovamente la lista di query, o di terminare la sessione.

In particolare, per rendere il sistema flessibile le query 1 e query 4 sono state parametrizzate (sfruttando la funzione PQexecParams) in modo da dare libertà all'utente di ricercare i dati desiderati:

- **Query 1:** è possibile specificare lo username dell'utente di cui si desidera visualizzare lo storico dei download (i valori consigliati per il testing sono ad esempio marco_92 o alex_gamer).
- **Query 4:** è possibile specificare dinamicamente il numero minimo di recensioni che un'app deve avere per entrare in classifica, e il limite massimo di risultati da mostrare (es. una Top 5 per app con almeno 1 recensione).

Sono state definite inoltre alcune funzioni di supporto orientate a migliorare l'esperienza utente e l'estetica dell'output a terminale, implementando l'uso di sequenze ANSI per i colori. Per poter stampare i risultati delle query in maniera chiara, intabellata e responsiva, sono state definite le funzioni:

- void stampa_separatore(int* larghezza_colonne, int colonne, char tratto, char giuntura);
- void stampa_query(PGresult* res);

Queste funzioni analizzano preventivamente il risultato della query per calcolare dinamicamente la larghezza massima necessaria per ciascuna colonna (confrontando la lunghezza del nome dell'attributo e dei valori delle tuple estratte). In questo modo, viene generata una tabella formattata su misura, priva di disallineamenti testuali.

La funzione principale che si occupa del ciclo di vita del programma è:

- PGresult* seleziona_query(PGconn* connessione);

Essa gestisce il menu utilizzabile dall'utente, esegue il parsing degli input, riceve gli eventuali parametri e chiama le funzioni di *execute* appropriate, ossia:

- PGresult* esegui_query(PGconn* connessione, const char* query);
- PGresult* esegui_query_parametrica(PGconn* connessione, const char* query, int numero_parametri, const char* const *parametri);

Queste si occupano dell'invio dei comandi a PostgreSQL e del controllo sulla correttezza dello stato restituito (mediante check con PGRES_TUPLES_OK).

Il main del programma, infine, dopo la fase di setup della connessione richiama in un ciclo while(1) la funzione di selezione e stampa, e continua ad eseguire le operazioni selezionate finché l'utente non sceglie di eseguire l'opzione di uscita per disconnettendosi dal DB tramite PQfinish.